An Introduction to Retrieval-Augmented Generation (RAG)

Víctor Macêdo
Expected duration: 60 minutes

Goals and non goals

Goals

- Explain what is RAG and the motivation for that
- We will focus on textual data
- Demo and explain the engineering aspects of RAG

Non goals

- Deep dive on RAG/Language models theory
- Multimodal RAG

Problem

How can LLMs access up-to-date knowledge?

https://www.visualcapitalist.com/the-surging-cost-of-training-ai-models/

Possible solutions

- Retrain or train a new model

  Too expensive, not feasible.

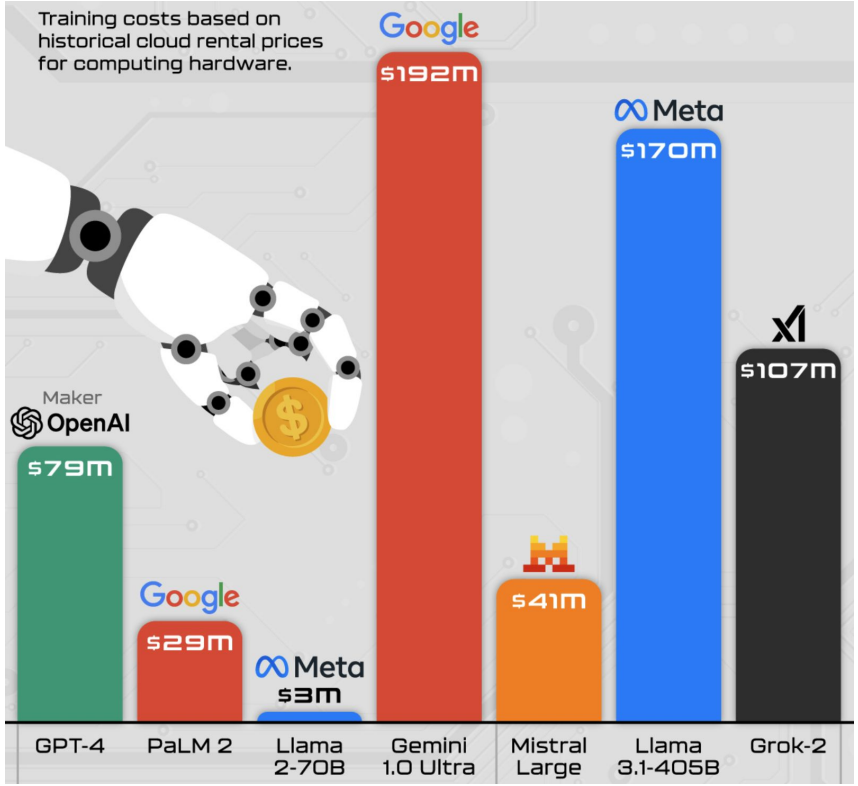
- Fine tune the model

  Useful to teach new concepts to the model but not feasible if new data is often needed.

- Use an external knowledge source

  That's RAG and the solution.

Cost of training Large Language Models



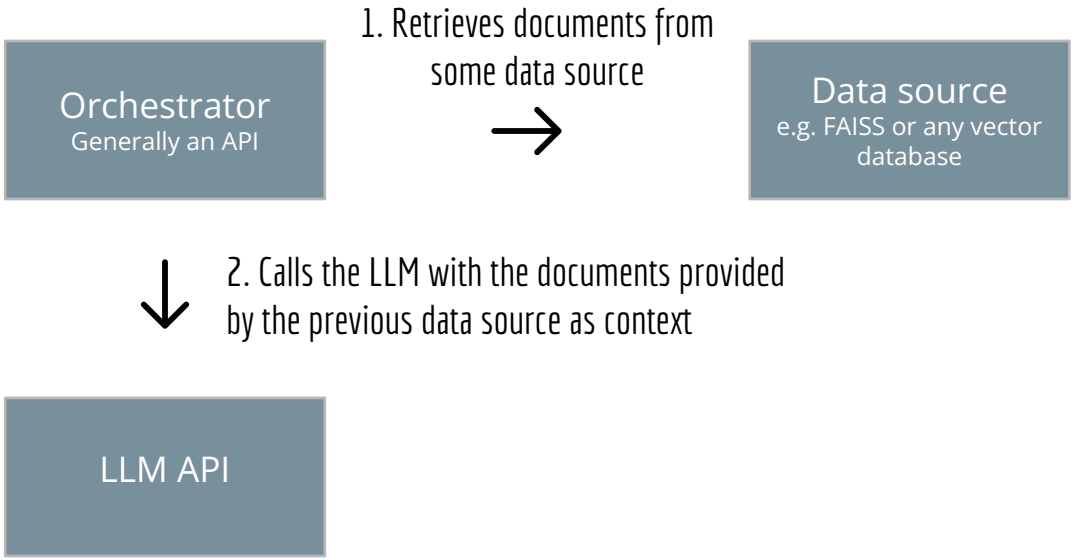
https://www.visualcapitalist.com/the-surging-cost-of-training-ai-models/

What is RAG

Retrieval-Augmented Generation (RAG) is a technique that enhances a language model by retrieving relevant information from external knowledge sources and using it as context to generate more accurate and up-to-date responses.

Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.

RAG + LLM basic workflow



Vector databases and Embeddings

- Vector databases store embeddings and retrieve the most similar ones using similarity metrics such as cosine similarity.

  Examples: PostgreSQL + pgvector, MongoDB and MySQL

- Embeddings are vectors that represent the semantic meaning of data in a numerical space.

  Example: [0.12, -0.45, 0.88, ...]

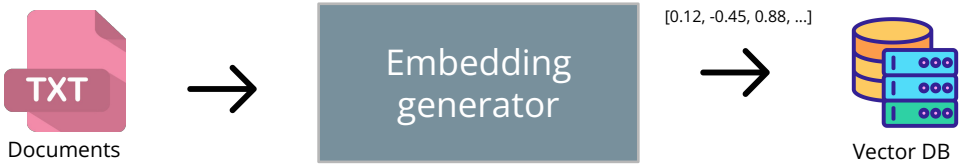
Mikolov et al., *Efficient Estimation of Word Representations in Vector Space*, ICLR 2013.

How to generate embeddings

Use a sentence embedding model to convert text into a vector that captures its semantic meaning.

Example: **all-MiniLM-L6-v2**

General workflow to populate a vector DB



Text Chunking

Problem

LLMs have limited context window, i.e., the amount of text it can receive as input. How do you split your text to create documents that are: relevant, meaningful and small enough?

Chunking strategies

- Fixed-size chunking: Split text into equal-length pieces
- Sentence-based chunking: Split text at sentence boundaries.
- Paragraph-based chunking: Split text by paragraphs.
- Semantic-based chunking: Use embeddings or NLP methods to split based on meaning (e.g. algorithms using Rhetorical Structure Theory).

General workflow to read from a vector DB

Query, e.g.: **Mercedes GLE 63 turbo model**

Demo

Problem

- I want a personal financial model to give me risk information about publicly listed companies.

Demo - Architecture - Index generation

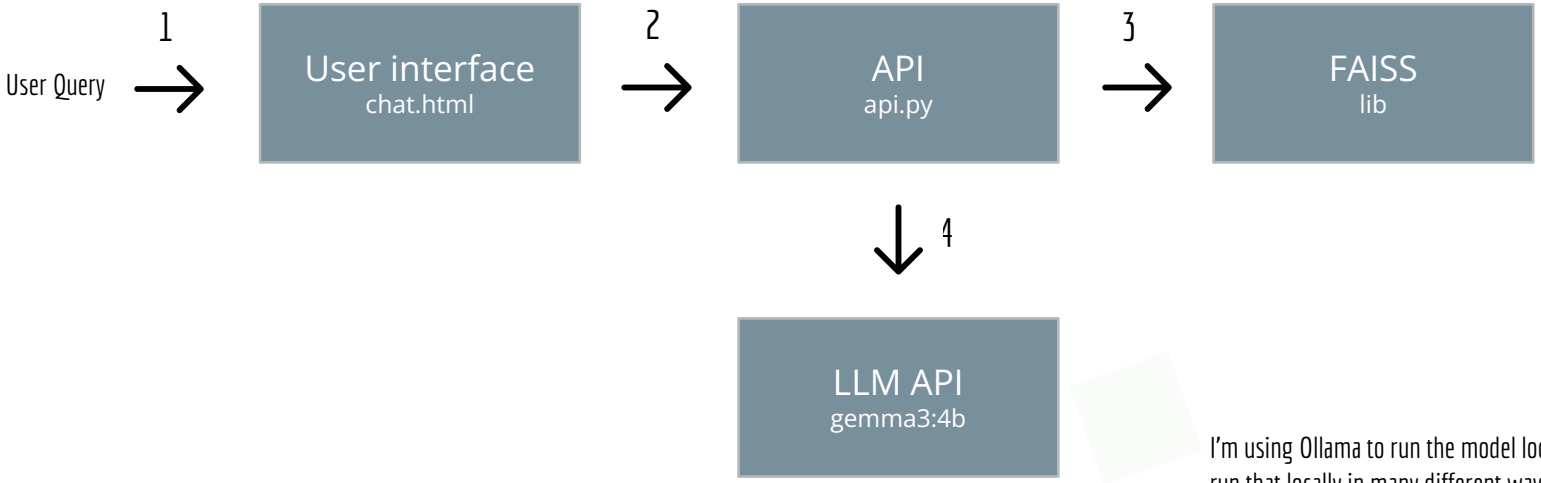
all-MiniLM-L6-v2 is the sentence embedding model used here because it's very lightweight and still performs well

What is FAISS?

*"Faiss is a library for efficient similarity search and clustering of dense vectors." **

* https://github.com/facebookresearch/faiss

Demo - Architecture - General



I'm using Ollama to run the model locally, you can use some external API or run that locally in many different ways.

Open Challenges

- The SOTA on text chunking are semantic aware methods, can we make that cheaper and better in terms F1 score?

- How to do multi-hop reasoning with RAG? there are attempts to solve the problem, but it's still far from a good solution.

- RAG in scale is expensive, how to make that faster and cheaper?

Questions?

Contact

ovictormacedo@gmail.com

Code and Slide

https://github.com/ovictormacedo/fin-rag-experiment/